

AMRITA VISHWA VIDYAPEETHAM CHENNAI
CAMPUS

Amrita School of Engineering

Department of ECE

Coding Practice

Activity Sheets - 03

(For the Batch 2023-2027)

Name:

Roll No.:

Department:

Instructions

The **Mock Test syllabus** for CSE CT | CYS | AIE | RAI program is enclosed in the attached document for your reference.

Materials for the Mock Test [Must-DO Coding Questions] is provided in a link.

It is mandatory to upload the **completed Activity sheets** before the deadline in the MS-**Form** link shared by your class Advisor.

Activity Sheet: 03 - Coding Questions from Strings – Part 1 [Easy + Medium + Hard Questions]

Each Activity Sheet contains total of **20 Questions**.

Platform: Any Online Compiler of your choice – **GDB Compiler**

Language: C | C++ | JAVA | PYTHON

The Activity Sheet will have the coding Questions and the Test Case. Once executed in Online Compiler, take **screen shot of the code and the output** and paste it in the space provided in the activity sheet.

The Final Activity Sheet should be in **PDF Format** and the file name should be your **roll number**.

Course Instructors: CARE TEAM

Dr.R.Annamalai Assistant Professor (Sl.Grade) | CSE-AIE

Dr.J.Umameswaran Assistant Professor (Sr.Grade) | CSE-CT

Ms.D. Sasikala Assistant Professor | CSE-AIE

Mr.NirmalRaj Assistant Professor | CSE-CYS

1. Given two strings A and B. Find the minimum number of steps required to transform string A into string B. The only allowed operation for the transformation is selecting a character from string A and inserting it in the beginning of string A.

Example 1:**Input:**

A = "abd"

B = "bad"

Output: 1

Explanation: The conversion can take place in 1 operation: Pick 'b' and place it at the front.

Example 2:**Input:**

A = "GeeksForGeeks"

B = "ForGeeksGeeks"

Output: 3

Explanation: The conversion can take place in 3 operations: Pick 'r' and place it at the front.

A = "rGeeksFoGeeks"

Pick 'o' and place it at the front.

A = "orGeeksFGeeks"

Pick 'F' and place it at the front.

A = "ForGeeksGeeks"

Your Task:

You don't need to read input or print anything. Complete the function transform() which takes two strings A and B as input parameters and returns the minimum number of steps required to transform A into B. If transformation is not possible return -1.

Expected Time Complexity: $O(N)$ where N is $\max(\text{length of A}, \text{length of B})$

Expected Auxiliary Space: $O(1)$

Constraints:

$1 \leq A.\text{length}(), B.\text{length}() \leq 104$

Code:

Output:

2. Given two non-empty strings $s1$ and $s2$, consisting only of lowercase English letters, determine whether they are anagrams of each other or not.

Two strings are considered anagrams if they contain the same characters with exactly the same frequencies, regardless of their order.

Examples:

Input: $s1 = \text{"geeks"}$ $s2 = \text{"kseeg"}$

Output: true

Explanation: Both the string have same characters with same frequency. So, they are anagrams.

Input: $s1 = \text{"allergy"}$, $s2 = \text{"allergy"}$

Output: false

Explanation: Although the characters are mostly the same, $s2$ contains an extra 'y' character. Since the frequency of characters differs, the strings are not anagrams.

Input: $s1 = \text{"listen"}$, $s2 = \text{"lists"}$

Output: false

Explanation: The characters in the two strings are not the same — some are missing or extra. So, they are not anagrams.

Constraints:

$1 \leq s1.size(), s2.size() \leq 105$

$s1, s2$ consists of lowercase English letters.

Code:

Output

3. Find the count of all possible strings of size n . Each character of the string is either 'R', 'B' or 'G'. In the final string there needs to be at least r number of 'R', at least b number of 'B' and at least g number of 'G' (such that $r + g + b \leq n$).

Example 1:

Input: $n = 4, r = 1, g = 1, b = 1$

Output: 36

Explanation: No. of 'R' ≥ 1 , No. of 'G' ≥ 1 , No. of 'B' ≥ 1 and (No. of 'R') + (No. of 'B') + (No. of 'G') = n then following cases are possible:

1. RBGR and its 12 permutation
2. RRGB and its 12 permutation
3. RBGG and its 12 permutation

Hence answer is 36.

Example 2:

Input: $n = 4, r = 2, g = 0, b = 1$

Output: 22

Explanation: No. of 'R' ≥ 2 , No. of 'G' ≥ 0 , No. of 'B' ≥ 1

and (No. of 'R') + (No. of 'B') + (No. of 'G') $\leq n$ then following cases are possible:

1. RRBR and its 4 permutation
2. RRBG and its 12 permutation
3. RRBB and its 6 permutation

Hence answer is 22.

Your Task:

You don't need to read input or print anything. Complete the function `possibleStrings()` which takes n, r, g, b as input parameter and returns the count of number of all possible strings..

Expected Time Complexity: $O(n^2)$

Expected Auxiliary Space: $O(n)$

Constraints:

$1 \leq n \leq 20$

Topic: Strings (Easy + Medium + Hard Coding Questions]

$1 \leq r+b+g \leq n$

Code:

Output

4. Given a valid expression containing only binary operators '+', '-', '*', '/' and operands, remove all the redundant parenthesis.

A set of parenthesis is said to be redundant if, removing them, does not change the value of the expression.

Note: The operators '+' and '-' have the same priority. '*' and '/' also have the same priority. '*' and '/' have more priority than '+' and '-'.

Example 1:

Input:

Exp = (A*(B+C))

Output: A*(B+C)

Explanation: The outermost parenthesis are redundant.

Example 2:

Input:

Exp = A+(B+(C))

Output: A+B+C

Explanation: All the parenthesis are redundant.

Your Task:

You don't need to read input or print anything. Your task is to complete the function removeBrackets() which takes the string Exp as input parameters and returns the updated expression.

Expected Time Complexity: O(N)

Expected Auxiliary Space: O(N)

Constraints:

1 < Length of Exp < 105

Exp contains uppercase english letters, '(', ')', '+', '-', '*' and '/'.

Code:

Output

5. Find the number of unique words consisting of lowercase alphabets only of length N that can be formed with at-most K contiguous vowels.

Example 1:

Input:

N = 2

K = 0

Output:

441

Explanation:

Total of 441 unique words are possible of length 2 that will have K(=0) vowels together, e.g. "bc", "cd", "df", etc are valid words while "ab" (with 1 vowel) is not a valid word.

Example 2:

Input:

N = 1

K = 1

Output:

26

Explanation:

All the english alphabets including vowels and consonants; as atmost K(=1) vowel can be taken.

Your Task:

You don't need to read input or print anything. Your task is to complete the function `kvowelwords()` which takes an Integer N, an integer K and returns the total number of words of size N with atmost K vowels. As the answer maybe too large please return answer modulo 1000000007.

Expected Time Complexity: $O(N \cdot K)$

Expected Auxiliary Space: $O(N \cdot K)$

Constraints:

$1 \leq N \leq 1000$

$0 \leq K \leq N$

Code:

Output

6. Problem Statement – Given a string S(input consisting) of '*' and '#'. The length of the string is variable.

The task is to find the minimum number of '*' or '#' to make it a valid string. The string is considered valid if the number of '*' and '#' are equal. The '*' and '#' can be at any position in the string.

Note : The output will be a positive or negative integer based on number of '*' and '#' in the input string.

- (*>#): positive integer
- (#>*): negative integer
- (#=*): 0

Code:

Output

7. Given a string `s`, reverse the string without reversing its individual words. Words are separated by dots(.).

Note: The string may contain leading or trailing dots(.) or multiple dots(.) between two words. The returned string should only have a single dot(.) separating the words, and no extra dots should be included.

Examples :

Input: `s = "i.like.this.program.very.much"`

Output: `"much.very.program.this.like.i"`

Explanation: The words in the input string are reversed while maintaining the dots as separators, resulting in `"much.very.program.this.like.i"`.

Input: `s = "..geeks..for.geeks."`

Output: `"geeks.for.geeks"`

Explanation: After removing extra dots and reversing the whole string, the input string becomes `"geeks.for.geeks"`.

Input: `s = "..home ...."`

Output: `"home"`

Explanation: The input string contains only one word with extra dots around it. After removing the extra dots, the output is `"home"`.

Constraints:

$1 \leq s.length() \leq 106$

String `s` contains only lowercase English alphabets and dots.

Code:

Output

8. A City Bus is a Ring Route Bus which runs in circular fashion. That is, Bus once starts at the Source Bus Stop, halts at each Bus Stop in its Route and at the end it reaches the Source Bus Stop again.

If there are n number of Stops and if the bus starts at Bus Stop 1, then after n th Bus Stop, the next stop in the Route will be Bus Stop number 1 always.

If there are n stops, there will be n paths. One path connects two stops. Distances (in meters) for all paths in Ring Route is given in array `Path[]` as given below:

`Path = [800, 600, 750, 900, 1400, 1200, 1100, 1500]`

Fare is determined based on the distance covered from source to destination stop as Distance between Input Source and Destination Stops can be measured by looking at values in array `Path[]` and fare can be calculated as per following criteria:

- If $d = 1000$ metres, then fare = 5 INR
- (When calculating fare for others, the calculated fare containing any fraction value should be ceiled. For example, for distance 900m when fare initially calculated is 4.5 which must be ceiled to 5)

`Path` is circular in function. Value at each index indicates distance till current stop from the previous one. And each index position can be mapped with values at same index in `BusStops []` array, which is a string array holding abbreviation of names for all stops as-

`"THANERAILWAYSTN" = "TH"`, `"GAONDEVI" = "GA"`, `"ICEFACTROY" = "IC"`,
`"HARINIWASCIRCLE" = "HA"`, `"TEENHATHNAKA" = "TE"`, `"LUISWADI" = "LU"`,
`"NITINCOMPANYJUNCTION" = "NI"`, `"CADBURRYJUNCTION" = "CA"`

Given, $n=8$, where n is number of total `BusStops`.

`BusStops = [ "TH", "GA", "IC", "HA", "TE", "LU", "NI", "CA" ]`

Write a code with function `getFare(String Source, String Destination)` which take Input as source and destination stops (in the format containing first two characters of the Name of the Bus Stop) and calculate and return travel fare.

Code:

Output

9. A new alien language uses the English alphabet, but the order of letters is unknown. You are given a list of words[] from the alien language's dictionary, where the words are claimed to be sorted lexicographically according to the language's rules.

Your task is to determine the correct order of letters in this alien language based on the given words. If the order is valid, return a string containing the unique letters in lexicographically increasing order as per the new language's rules. If there are multiple valid orders, return any one of them.

However, if the given arrangement of words is inconsistent with any possible letter ordering, return an empty string ("").

A string a is lexicographically smaller than a string b if, at the first position where they differ, the character in a appears earlier in the alien language than the corresponding character in b. If all characters in the shorter word match the beginning of the longer word, the shorter word is considered smaller.

Note: Your implementation will be tested using a driver code. It will print true if your returned order correctly follows the alien language's lexicographic rules; otherwise, it will print false.

Examples:

Input: words[] = ["baa", "abcd", "abca", "cab", "cad"]

Output: true

Explanation: A possible correct order of letters in the alien dictionary is "bdac".

The pair "baa" and "abcd" suggests 'b' appears before 'a' in the alien dictionary.

The pair "abcd" and "abca" suggests 'd' appears before 'a' in the alien dictionary.

The pair "abca" and "cab" suggests 'a' appears before 'c' in the alien dictionary.

The pair "cab" and "cad" suggests 'b' appears before 'd' in the alien dictionary.

So, 'b' → 'd' → 'a' → 'c' is a valid ordering.

Input: words[] = ["caa", "aaa", "aab"]

Output: true

Explanation: A possible correct order of letters in the alien dictionary is "cab".

The pair "caa" and "aaa" suggests 'c' appears before 'a'.

The pair "aaa" and "aab" suggests 'a' appear before 'b' in the alien dictionary.

So, 'c' → 'a' → 'b' is a valid ordering.

Input: words[] = ["ab", "cd", "ef", "ad"]

Output: ""

Explanation: No valid ordering of letters is possible.

The pair "ab" and "ef" suggests 'a' appears before 'e'.

The pair "ef" and "ad" suggests 'e' appears before 'a', which contradicts the ordering rules.

Constraints:

$1 \leq \text{words.length} \leq 500$

$1 \leq \text{words}[i].\text{length} \leq 100$

words[i] consists only of lowercase English letters.

Code:

Output

10. Jack and Jill are playing a string game. Jack has given Jill two strings A and B. Jill has to derive a string C from A, by deleting elements from string A, such that string C does not contain any element of string B. Jill needs help to do this task. She wants a program to do this as she is lazy. Given strings A and B as input, give string C as Output.

Code:

Output

11. Given two strings *s* and *p*. Find the smallest substring in *s* consisting of all the characters (including duplicates) of the string *p*. Return empty string in case no such substring is present. If there are multiple such substring of the same length found, return the one with the least starting index.

Examples:

Input: *s* = "timetopractice", *p* = "toc"

Output: "toprac"

Explanation: "toprac" is the smallest substring in which "toc" can be found.

Input: *s* = "zoomlazapzo", *p* = "oza"

Output: "apzo"

Explanation: "apzo" is the smallest substring in which "oza" can be found.

Input: *s* = "zoom", *p* = "zooe"

Output: ""

Explanation: No substring is present containing all characters of *p*.

Constraints:

$1 \leq s.length(), p.length() \leq 106$

s, *p* consists of lowercase english letters

Code:

Output

12. Given two strings s_1 and s_2 . Return the minimum number of operations required to convert s_1 to s_2 . The possible operations are permitted:

Insert a character at any position of the string.

Remove any character from the string.

Replace any character from the string with any other character.

Examples:

Input: $s_1 = \text{"geek"}, s_2 = \text{"gesek"}$

Output: 1

Explanation: One operation is required, inserting 's' between two 'e' in s_1 .

Input: $s_1 = \text{"gfg"}, s_2 = \text{"gfg"}$

Output: 0

Explanation: Both strings are same.

Input: $s_1 = \text{"abcd"}, s_2 = \text{"bcfe"}$

Output: 3

Explanation: We can convert s_1 into s_2 by removing 'a', replacing 'd' with 'f' and inserting 'e' at the end.

Constraints:

$1 \leq s_1.length(), s_2.length() \leq 10^3$

Both the strings are in lowercase.

Code:

Output

13. Given a string `str` of opening and closing brackets '(' and ')' only. The task is to find an equal point. An equal point is an index (0-based) such that the number of closing brackets on the right from that point must be equal to the number of opening brackets before that point.

Examples:

Input: `str = "((()))(`

Output: 4

Explanation: After index 4, string splits into `(( ))` and `))(`. Number of opening brackets in the first part is equal to number of closing brackets in the second part.

Input : `str = "))(`

Output: 2

Explanation: As after 2nd position i.e. `))` and "empty" string will be split into these two parts: So, in this number of opening brackets i.e. 0 in the first part is equal to number of closing brackets in the second part i.e. also 0.

Expected Time Complexity: $O(n)$

Expected Auxiliary Space: $O(n)$

Constraints:

$1 \leq \text{str.size()} \leq 10^6$

`str` consists of only '(' and ')' brackets.

Code:

Output

14. Given two numbers as strings s1 and s2. Calculate their Product.

Note: The numbers can be negative and You are not allowed to use any built-in function or convert the strings to integers. There can be zeros in the beginning of the numbers. You don't need to specify '+' sign in the beginning of positive numbers.

Examples:

Input: s1 = "0033", s2 = "2"

Output: "66"

Explanation: $33 * 2 = 66$

Input: s1 = "11", s2 = "23"

Output: "253"

Explanation: $11 * 23 = 253$

Input: s1 = "123", s2 = "0"

Output: "0"

Explanation: Anything multiplied by 0 is equal to 0.

Constraints:

$1 \leq s1.size() \leq 103$

$1 \leq s2.size() \leq 103$

Code:

Output

15. A message containing letters A-Z is being encoded to numbers using the following mapping:

'A' -> 1
'B' -> 2
...
'Z' -> 26

You are given a string `digits`. You have to determine the total number of ways that message can be decoded.

Examples:

Input: `digits = "123"`

Output: 3

Explanation: "123" can be decoded as "ABC"(1, 2, 3), "LC"(12, 3) and "AW"(1, 23).

Input: `digits = "90"`

Output: 0

Explanation: "90" cannot be decoded, as it's an invalid string and we cannot decode '0'.

Input: `digits = "05"`

Output: 0

Explanation: "05" cannot be mapped to "E" because of the leading zero ("5" is different from "05"), the string is not a valid encoding message.

Constraints:

$1 \leq \text{digits.size()} \leq 103$

Code:

Output

16. Given a string `str` of length `n`, find if the string is K-Palindrome or not. A k-palindrome string transforms into a palindrome on removing at most `k` characters from it.

Example 1:

Input: `str = "abcdecba"`

`n = 8, k = 1`

Output: 1

Explanation: By removing 'd' or 'e' we can make it a palindrome.

Example 2:

Input: `str = "abcdefcba"`

`n = 9, k = 1`

Output: 0

Explanation: By removing a single character we cannot make it a palindrome.

Your Task:

You do not need to read input or print anything. Your task is to complete the function `kPalindrome()` which takes string `str`, `n`, and `k` as input parameters and returns 1 if `str` is a K-palindrome else returns 0.

Expected Time Complexity: $O(n*n)$

Expected Auxiliary Space: $O(n*n)$

Constraints:

$1 \leq n, k \leq 103$

Code:

Output

17. Given two strings txt and pat, return the 0-based index of the first occurrence of the substring pat in txt. If pat is not found, return -1.

Note: You are not allowed to use the inbuilt function.

Examples :

Input: txt = "GeeksForGeeks", pat = "Fr"

Output: -1

Explanation: "Fr" is not present in the string "GeeksForGeeks" as substring.

Input: txt = "GeeksForGeeks", pat = "For"

Output: 5

Explanation: "For" is present as substring in "GeeksForGeeks" from index 5 (0 based indexing).

Input: txt = "GeeksForGeeks", pat = "gr"

Output: -1

Explanation: "gr" is not present in the string "GeeksForGeeks" as substring.

Constraints:

$1 \leq \text{txt.size()}, \text{pat.size()} \leq 1000$

Code:

Output

18. Given a string `s` consisting of opening and closing parenthesis '(' and ')'. Find the length of the longest valid parenthesis substring.

A parenthesis string is valid if:

For every opening parenthesis, there is a closing parenthesis.

The closing parenthesis must be after its opening parenthesis.

Examples :

Input: `s = "((()"`

Output: 2

Explanation: The longest valid parenthesis substring is "()".

Input: `s = ")()())"`

Output: 4

Explanation: The longest valid parenthesis substring is "()()".

Input: `s = "(())"`

Output: 2

Explanation: The longest valid parenthesis substring is "()".

Constraints:

$1 \leq s.size() \leq 10^6$

`s` consists of '(' and ')' only

Code:

Output

19. Given a string s and a string dictionary d , find the longest string in the dictionary that can be formed by deleting some characters of the given string. If there are more than one possible results, return the longest word with the smallest lexicographical order. If there is no possible result, return the empty string.

Examples :

Input: $d = \{"ale", "apple", "monkey", "plea"\}$, $s = "abpcplea"$

Output: "apple"

Explanation: After deleting "b", "c", "a" S became "apple" which is present in d .

Input: $d = \{"a", "b", "c"\}$, $s = "abpcplea"$

Output: "a"

Explanation: After deleting "b", "p", "c", "p", "l", "e", "a" S became "a" which is present in d .

Expected Time Complexity: $O(n \cdot x)$

Expected Auxiliary Space: $O(x)$ where x is the length of the string in d .

Constraints:

$1 \leq n, x \leq 103$

Code:

Output

20. Given strings s_1 , s_2 , and s_3 , find whether s_3 is formed by an interleaving of s_1 and s_2 . Interleaving of two strings s_1 and s_2 is a way to mix their characters to form a new string s_3 , while maintaining the relative order of characters from s_1 and s_2 . Conditions for interleaving:

Characters from s_1 must appear in the same order in s_3 as they are in s_1 .

Characters from s_2 must appear in the same order in s_3 as they are in s_2 .

The length of s_3 must be equal to the combined length of s_1 and s_2 .

Examples :

Input: $s_1 = \text{"AAB"}$, $s_2 = \text{"AAC"}$, $s_3 = \text{"AAAABC"}$

Output: true

Explanation: The string "AAAABC" has all characters of the other two strings and in the same order.

Input: $s_1 = \text{"AB"}$, $s_2 = \text{"C"}$, $s_3 = \text{"ACB"}$

Output: true

Explanation: s_3 has all characters of s_1 and s_2 and retains order of characters of s_1 .

Input: $s_1 = \text{"YX"}$, $s_2 = \text{"X"}$, $s_3 = \text{"XXY"}$

Output: false

Explanation: "XXY" is not interleaved of "YX" and "X". The strings that can be formed are YXX and XYX

Constraints:

$1 \leq s_1.\text{length}, s_2.\text{length} \leq 300$

$1 \leq s_3.\text{length} \leq 600$

Code:

Output